Frontend / Backend Coupling

How the two parts of a fullstack application depend on each other

KIIL @ EK, PROG. 2

Today's agenda

- what the frontend and backend each do
- how they communicate through HTTP and JSON
- what coupling means in practice
- what an API contract is
- how a contract helps both code and teamwork

Frontend and backend: different responsibilities

Part	Main responsibility
Frontend	UI, user input, rendering data
Backend	Data, logic, request handling
Browser	Runs JavaScript and sends requests
API	Agreement between frontend and backend

Separation of concerns

A useful design principle is **separation of concerns**.

That means:

- the frontend should focus on presentation and interaction
- the backend should focus on data and application logic
- the communication between them should be explicit

This makes the system easier to build, test, and change.

What is coupling?

Coupling means that one part of the system depends on another.

In our case:

- the frontend depends on the backend returning expected data
- the backend depends on the frontend sending valid requests

Some coupling is unavoidable. The problem is **unclear coupling**.

One complete request / response cycle

1. The user selects e.g. some pizza ingredients in the browser
2. JavaScript reads the selected IDs
3. The frontend sends a request to the backend
4. The backend receives the request
5. The backend finds matching pizza recipe
6. The backend returns JSON
7. The frontend parses the JSON
8. The frontend renders the result

> This is the path the data takes through the system.

Steps 1-2: user input and frontend state

```html
<select id="pizzaIngredientSelect" multiple>
  <option value="1">Tomato sauce</option>
  <option value="2">Mozzarella</option>
</select>
```

```javascript
const select = document.querySelector("#pizzaIngredientSelect");
const selectedIds = Array.from(select.selectedOptions)
  .map(option => Number(option.value));
```

What happens here:

- the browser shows a form element
- the user chooses one or more values
- JavaScript reads those values
- the selected ingredients now exist as data in the frontend

Step 3: the frontend sends the request

```
const url = `/api/suggestions?ingredientIds=${selectedIds.join(",")}`;
const response = await fetch(url);
```

```
GET /api/suggestions?ingredientIds=1,2
```

What happens here:

- JavaScript turns local state into a URL
- the browser sends an HTTP request
- this request is the frontend asking the backend for data

Step 4: the backend receives the request

```
@GetMapping("/api/suggestions")
public List<SuggestionDto> getPizzaSuggestions(
    @RequestParam List<Long> ingredientIds
) {
    return suggestionService.getPizzaSuggestions(ingredientIds);
}
```

What happens here:

- Spring receives the request
- `ingredientIds` are read from the query string
- the controller passes the work on to a service
- the controller should stay thin and focused on HTTP

Step 5: the backend finds matching pizza recipes

```
public List<SuggestionDto> getPizzaSuggestions(List<Long> ingredientIds) {
    return recipeRepository.findAll().stream()
        .map(recipe -> toSuggestion(recipe, ingredientIds))
        .sorted(Comparator.comparingDouble(SuggestionDto::score).reversed())
        .toList();
}
```

What happens here:

- the service gets the selected ingredient IDs
- it compares them with stored pizza recipes
- it calculates a result that can be sent back
- this is backend work

Step 6: the backend builds the response

DTO in Java

```java
public record SuggestionDto(
    Long recipeId,
    String recipeName,
    List<String> matchedIngredients,
    List<String> missingIngredients,
    double score
) {}
```

JSON sent back

```json
[
  {
    "recipeId": 1,
    "recipeName": "Margherita",
    "matchedIngredients": ["Tomato sauce", "Mozzarella"],
    "missingIngredients": ["Pizza dough"],
    "score": 0.67
  }
]
```

Spring converts Java objects into JSON.
That JSON is what the frontend must understand.

Steps 7-8: the frontend parses and renders

```
if (!response.ok) {
  throw new Error("Could not load suggestions");
}

const suggestions = await response.json();
renderSuggestions(suggestions);
```

```
function renderSuggestions(suggestions) {
  suggestionsContainer.innerHTML = suggestions
    .map(s => `<li>${s.recipeName} (${s.score})</li>`)
    .join("");
}
```

What happens here:

- the frontend checks whether the request succeeded
- it parses the JSON into JavaScript data
- it renders the result into the page

What is an API contract?

An API contract is the agreement about:

- endpoint path
- HTTP method
- request parameters or request body
- response shape
- field names
- data types
- status codes

A contract is about structure and meaning

Part of the contract	What it tells us
Endpoint path	Which backend function we are calling
HTTP method	Whether we are reading, creating, or updating data
Field names	What keys the frontend can rely on
Data types	Whether a value is a string, number, boolean, list, and so on
Meaning	What the values actually represent in the application

A good contract is not only about shape, but also about meaning.

A contract can be written in different forms

Common ways to define it:

- example JSON in a README, ideally with a short contract table
- task description with request/response examples (to supplement the contract)
- Postman collection (to supplement the contract)
- OpenAPI / Swagger (usually for larger projects)

At minimum, it should be clear what comes in and what goes out.

Example: a small but complete contract

Success

```
200 OK
```

Request

```
GET /api/suggestions?ingredientIds=1,2
```

Response

```
[
  {
    "recipeId": 1,
    "recipeName": "Margherita",
    "matchedIngredients": ["Tomato sauce", "Mozzarella"],
    "missingIngredients": ["Basil"],
    "score": 0.67
  }
]
```


Error example

Contract example with request body

Not every request uses query parameters.

A `POST` contract might look like this:

```
POST /api/pizza-orders
Content-Type: application/json
```

```
{
  "pizzaId": 2,
  "tableNumber": 4
}
```

And the response might be:

```
{
  "id": 4,
  "pizzaId": 2,
  "pizzaName": "Pepperoni",
  "tableNumber": 4,
  "status": "NEW"
}
```

The contract is also a team agreement

A contract helps a frontend team and backend team work at the same time.

Example workflow:

- backend team agrees to return a certain JSON shape
- frontend team builds the page using mock data in that shape
- later the mock data is replaced with real `fetch`

Before implementation, teams should agree on:

- endpoint
- method
- request format
- response format
- important field names
- data types
- status codes and error cases

Why IDs matter in a contract

If we identify data by name, we can get problems:

- names can change
- names can be duplicated
- names are meant for display, not identity
- updates become harder to reason about

Example:

```
{
  "ingredientName": "Tomato"
}
```

A better contract is:

```
{
  "ingredientId": 1
}
```

Example of a broken contract

Frontend expects

```
recipe.recipeName
recipe.score
recipe.missingIngredients
```

Backend returns

```
{
  "name": "Margherita",
  "matchScore": 0.67,
  "missing": ["Basil"]
}
```

What happens?

- `recipe.recipeName` becomes `undefined`
- `recipe.score` becomes `undefined`
- `recipe.missingIngredients` is missing
- the page may show nothing or break

Both sides may look fine, but put together it fails.

How to create a contract in practice

A simple method for student projects:

1. write the endpoint
2. choose the method
3. write one sample request
4. write one sample response
5. choose field names carefully
6. let both frontend and backend code follow that shape

Why we start with hardcoded data

Hardcoded data is useful because it separates two problems:

- can the frontend render the right shape?
- can the frontend connect to the backend correctly?

This gives a safer order:

1. mock data
2. real `fetch`
3. mutations

Contract problems

When a feature breaks, check:

- was the request sent?
- was the URL correct?
- did the backend respond?
- what JSON came back?
- does the frontend code match those field names?